

Project Proposal for the FISI Project in Learning Field 10

Class: IT4o / IT4k · **Group:** FISI DV 14

Members: Dissy Ragouena, Jamen Werneke

1. Project Title

Automated provisioning and configuration of Linux web servers and a PostgreSQL database using Ansible as configuration management software

1.1 Short Form of the Task

The chosen Learning Field 10, Scenario A, requires finding and working on a free project in the subject area *"Providing server services and automating administration tasks"*. This project fulfills that requirement by replacing the manual setup of a web and database tier with a fully automated, repeatable Ansible deployment.

1.2 As-Is Analysis

At the time of this proposal, Linux servers in the existing environment are created and configured manually. Every step — installing the web server, adjusting its configuration, installing and initializing the database, and placing the application content — is carried out by hand over an SSH session by IT staff.

This approach has several disadvantages:

- **Repetitive effort:** every additional server requires the same sequence of steps to be repeated, consuming working hours for work that produces no new value.
- **Configuration drift:** manually configured servers are never truly identical. Small deviations accumulate and lead to errors that are difficult to reproduce and diagnose.
- **Lack of documentation:** the actual state of a server exists only on the server itself. There is no versioned, readable description of how it was built.
- **Poor scalability:** provisioning three identical web servers takes three times as long as provisioning one.
- **Long recovery time:** if a server is lost, it must be rebuilt manually from memory or from incomplete notes.

Particularly in view of the shortage of skilled workers, it is necessary for companies to automate this type of task.

2. Objective / Development of the Target Concept

2.1 What is to be achieved by the end of the project?

Through the use of the configuration management software Ansible, a complete two-tier infrastructure is to be provisioned from a single command. The target state comprises:

- **three identical web servers** running nginx, each delivering the same page containing the text *"This server is handled by Ansible"*
- **one database server** running PostgreSQL, which stores an image that is displayed on that page

The entire configuration is described declaratively in version-controlled files. The desired state of the systems is therefore defined in code rather than in the memory of the administrator. Adding a fourth web server requires a single line in the inventory and no change to the logic.

The key property to be demonstrated is **idempotency**: running the playbook repeatedly produces no changes if the systems already match the desired state. This is the fundamental difference between configuration management and a conventional shell script, and it makes the automation safe to execute at any time.

The expected benefits are the reduction of repetitive manual work and therefore of costs, improved scalability, guaranteed consistency across all servers of the same role, and a significantly shorter recovery time in the event of a failure, since a lost server can be rebuilt by re-running the playbook.

2.2 Which requirements must be fulfilled?

The following requirements are to be met by the project:

- Fully automated installation and configuration of nginx on all web hosts
- Fully automated installation and configuration of PostgreSQL on the database host
- Storage of a binary image inside the database and its delivery via the website
- Identical, reproducible configuration across all web servers, generated from templates
- Central variable management, so that ports, names and content are changed in exactly one place
- Idempotent execution of the entire playbook
- Reusable, self-contained roles for the web and database tiers
- A rollback mechanism enabling the changes to be reverted for repeated testing
- Verifiability of the result through a documented before/after comparison
- Extensibility of the configuration to accommodate future requirements

2.3 Which constraints must be taken into account?

- **Agentless architecture**: Ansible communicates exclusively over SSH. The target systems must therefore be reachable via SSH, provide Python 3, and offer an account with sudo privileges.
- **Prescribed syntax**: playbooks, roles and inventories must follow the YAML structure and module semantics defined by Ansible. Deviating from these conventions is not possible.
- **Tool commitment**: the choice of Ansible cannot be reversed without considerable effort, since all configurations would have to be rewritten for another tool.

- **Available hardware:** the target platform is a Server running Debian. Its resources are limited.
- **Distribution dependency:** package names and service names differ between distribution families. The roles are developed for the Debian family.
- **Persistence of the image:** the image is transferred from the database to the web root at deployment time. A change to the image in the database therefore requires the playbook to be run again.

3. Project Planning

This is an in-house project. Short iteration cycles with feedback from members are planned, and the project is to be developed in an agile manner. The entire project is to be versioned with Git.

Phase	Task	Hours
Analysis		3 h
	Carry out as-is analysis	1 h
	Comparison of alternatives (Ansible / Puppet / Chef / SaltStack)	1 h
	Cost-effectiveness assessment	1 h
Design		4 h
	Technology selection	1 h
	Create component diagram and define the target architecture	1 h
	Create project proposal	2 h
Implementation		28 h
	Set up the environment (SSH keys, inventory, control node)	4 h
	Develop the PostgreSQL role (installation, database, table, image import)	7 h
	Develop the nginx role (installation, port configuration, templates)	7 h
	Implement the transfer of the image from the database to the web tier	4 h
	Testing, troubleshooting and verification of idempotency	4 h
	Develop the rollback playbook and the test script	2 h
Documentation & Acceptance		5 h
	Create presentation	2 h
	Acceptance meeting	1 h
	Maintain portfolio	2 h
Total		40 h

4. Technical Environment

Component	Selection
Configuration management	Ansible (agentless, YAML, push-based)
Control node	Laptop running Linux
Target system	Server running Linux
Web server	nginx
Database	PostgreSQL
Connectivity	SSH over Tailscale VPN
Versioning	Git

for the choice of Ansible: unlike Puppet or Chef, Ansible requires no agent on the target systems and no central master server. This makes it particularly suitable for a heterogeneous environment and for a project of this size. The declarative YAML syntax is readable without programming knowledge, which additionally serves the goal of self-documenting infrastructure.